Deploy

One container image runs both workloads:

- **the playground**, the caller simulator and admin console behind a sign-in (the default command)
- **the gate job**, `gate-job`, run nightly and by hand

Design and decisions: SPEC §9 and §10 (D-57, D-80 to D-85).

The image

Build it from the repo root. The build context is an allow-list in `.dockerignore` (`python/`, `fixtures/`, `tests/human/`); `.env` never enters it.

```
docker build -f deploy/Dockerfile -t pirtis .
```

Azure runs `linux/amd64`. On an Apple-silicon Mac, check that build too:

```
docker buildx build --platform linux/amd64 -f deploy/Dockerfile -t pirtis:amd64 --load .
```

Run the playground on http://127.0.0.1:8000, keeping calls in `python/var/`. The key is read from `python/.env` and never baked in. The image requires sign-in accounts; make one with `cd python && uv run python -m pirtis.playground.auth add <name>` (it writes `deploy/.users` and the password to `deploy/.passwords/<name>.txt`).

```
docker run --rm --env-file python/.env -e PIRTIS_USERS="$(paste -sd, deploy/.users)" -e
PIRTIS_SESSION_KEY="$(openssl rand -hex 32)" -v "$PWD/python/var:/data" -p 127.0.0.1:8000:8000
pirtis
```

Run the nightly gate (the given cases and the red team, 3 runs each) against the same data. It prints one JSON line and exits 1 if the gate fails.

```
docker run --rm --env-file python/.env -v "$PWD/python/var:/data" pirtis gate-job
```

`gate-job --suites all` runs every suite, like `make gate`; `--id a,b` narrows it to those cases, and `--no-extract` skips the extractor eval.

Variable	In the image	Meaning
`LLM_API_KEY`	not set: pass it at run time (a Container App secret in Azure)	the model key
`LLM_BASE_URL`	not set: the OpenAI default	Azure OpenAI's v1 endpoint, if used
`PIRTIS_HASH_SALT`	not set: a fixed development salt	the salt for phone hashes; set a secret one in Azure
`PIRTIS_DATA_DIR`	`/data`	where records are kept: the mounted file share in Azure

Variable	In the image	Meaning
`PIRTIS_PROMOTE_DIR`	empty	a promoted case is downloaded, not written into the image
`PIRTIS_USERS`	not set: the container refuses to start	sign-in accounts, `name:salt:hash` (made by `make user`)
`PIRTIS_SESSION_KEY`	not set: required with accounts	signs the session cookie; 32 characters or more
`PIRTIS_AUTH`	`required`	without accounts, refuse to start rather than run open
`PIRTIS_HOST`, `PORT`	`0.0.0.0`, `8000`	where the playground listens
`PIRTIS_RATE_PER_MIN`	not set: 30	units of model spending each account may use a minute
`PIRTIS_SPEND_PER_DAY`	not set: 2000	units for everyone together each UTC day (a unit is about one model call)
`PIRTIS_GATE_SUITES`	not set: `nightly`	the gate job's default suites

Azure

Everything is in `deploy/terraform/` (Terraform 1.5 or later, or OpenTofu: run with `TF=tofu`). It creates 15 resources in one resource group in an EU region:

- the base: Log Analytics (30 days, 1 GB a day at most), a Container Apps environment, a Basic registry, a managed identity that can only pull, and a storage account with one file share (1 GiB)
- the app (0 to 1 replicas) and the gate job (nightly at 03:00 UTC, no retries)
- an email action group and three alerts

Run these from `deploy/`, signed in with `az login`, in this order the first time. Terraform asks before it changes anything; add `APPROVE=-auto-approve` to skip that.

```
cp terraform/terraform.tfvars.example terraform/terraform.tfvars
```

Set `alert_email` in that file, then:

```
make infra
```

```
make storage-link
```

```
make image
```

```
make deploy
```

```
make secrets
```

```
make user NAME=ana
```

Target	What it does
`infra`	the base and the alerts (no app yet: there's no image)
`storage-link`	links the share to the environment; the account key goes from `az storage account keys list` straight into `az containerapp env storage set`, never into a file
`image`	builds for linux/amd64, pushes with an Entra token (no registry password), and writes the digest to `.image-digest`
`deploy`	the gate must hold for this agent version (a saved full pass, or `make gate` runs now), then the app and the job, by digest
`secrets`	the model key from `python/.env` (never printed); a hash salt and a session key, made once and kept after
`user NAME=…` / `user-remove NAME=…`	adds or removes a sign-in account and restarts the app; the password is in `.passwords/<name>.txt` (mode 600) to hand over, then delete
`url`, `logs`, `restart`	the app's address, its live console, a restart after a secret change
`rotate-session`	a new session key: everyone is signed out
`gate-now`	runs the gate job now; its line is in the logs, its run in the Reliability view
`calibrate`	judge calibration on the ratings saved in the cloud: one run of the job's image with the real key by secret reference (not yet run in the cloud: there are no ratings yet)
`alert-test`	fires the gate-regression alert end to end: the job with a key that can't work, so no tokens; its records go to `/tmp`
`destroy`	asks for the resource group's name, then removes everything

The state is a secret. Terraform's state holds the storage account's keys and, after any refresh, the Container App secrets' values. The Makefile writes it owner-only (`umask 077`); keep it local and out of git (it is ignored), or move it to a remote, encrypted backend.

Cost while it stands: about €5–6 a month, mostly the registry. The app scales to zero, and its first request after a quiet spell takes about 40 seconds. Each nightly gate spends about $0.06 of model credit.

Monitoring: `monitoring/queries.kql` holds the saved queries: intents, transfers and fallbacks, turns answered without the model, tokens, latency, agent errors and the gate history.